SKRIPSI

PERANGKAT LUNAK PEMBUAT VISUALISASI GRAF STRUKTUR DOM HALAMAN WEB

Bram Matthew

NPM: 6182101043

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS
UNIVERSITAS KATOLIK PARAHYANGAN
2026

DAFTAR ISI

Daftar Isi iii

Daftar Gambar v

1 Pendahuluan 1
- 1.1 Latar Belakang . . . 1
- 1.2 Rumusan Masalah . . . 1
- 1.3 Tujuan . . . 2
- 1.4 Batasan Masalah . . . 2
- 1.5 Metodologi . . . 2
- 1.6 Sistematika Pembahasan . . . 2

2 Landasan Teori 5
- 2.1 Hypertext Markup Language (HTML) . . . 5
  - 2.1.1 Pengertian HTML . . . 5
  - 2.1.2 Elemen dan Tag HTML . . . 5
  - 2.1.3 Struktur dasar HTML . . . 6
  - 2.1.4 Hierarki elemen HTML . . . 7
- 2.2 Document Object Model (DOM) . . . 8
  - 2.2.1 Pengertian DOM . . . 8
  - 2.2.2 Struktur DOM sebagai Tree . . . 8
  - 2.2.3 Jenis *node* pada DOM . . . 9
- 2.3 Jsoup . . . 10
  - 2.3.1 Pengertian Jsoup . . . 10
  - 2.3.2 Kelas `Jsoup` . . . 10
  - 2.3.3 Kelas `Document` . . . 11
  - 2.3.4 Kelas Element . . . 11
  - 2.3.5 Penggunaan Jsoup dalam Pemrosesan DOM . . . 11
- 2.4 Struktur Data Tree . . . 12
  - 2.4.1 Pengertian Tree . . . 12
  - 2.4.2 Tree pada Struktur DOM . . . 12
- 2.5 Struktur Data Graf . . . 13
  - 2.5.1 Pengertian Graf . . . 13
  - 2.5.2 Representasi Graf . . . 13
  - 2.5.3 Graf Berarah . . . 14
  - 2.5.4 Representasi DOM sebagai graf . . . 15
  - 2.5.5 Pengertian Traversal Graf . . . 15
  - 2.5.6 Depth-First Search . . . 15
- 2.6 Graphviz . . . 17
  - 2.6.1 Pengertian Graphviz . . . 17
  - 2.6.2 Format DOT . . . 17

3 Analisis 19

3.1 Analisis Masalah 19
3.1.1 Analisis Struktur HTML 19
3.1.2 Analisis Pembentukan DOM 19
3.1.3 Analisis Representasi DOM sebagai Graf 20
3.1.4 Analisis Traversal DOM 21
3.1.5 Analisis Kebutuhan Visualisasi 21
3.2 Analisis Kebutuhan Sistem 22
3.2.1 Analisis *Use Case* dan Skenario 22
3.2.2 Kebutuhan Non-Fungsional 24
3.3 Analisis Sistem Usulan 25
3.3.1 Gambaran Umum Sistem Usulan 25
3.3.2 Model Representasi Graf 26
3.3.3 Diagram Sekuens Sistem Usulan 26
3.3.4 Komponen Sistem Usulan 27
3.3.5 Analisis Masukan dan Keluaran Sistem 28

Daftar Referensi **29**

DAFTAR GAMBAR

2.1 Gambar Tag pembuka dan penutup 6
2.2 Contoh DOM Tree 9
2.3 Contoh graf berarah sederhana (*node* dan *edge*) 14
2.4 Ilustrasi proses DFS pada graf berarah 17
2.5 Visualisasi dari format DOT dengan menggunakan Graphviz 18

3.1 Diagram sekuens proses visualisasi graf struktur DOM 27

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi web yang sangat pesat menyebabkan halaman web modern memiliki struktur yang semakin kompleks. Setiap halaman web dibangun menggunakan *Hypertext Markup Language* (HTML) yang secara umum dapat direpresentasikan dalam bentuk struktur hierarkis yang dikenal sebagai *Document Object Model* (DOM). Struktur DOM ini berbentuk pohon (*tree*) yang terdiri dari elemen-elemen HTML yang saling terhubung melalui hubungan induk (*parent*) dan anak (*child*) [1].

Meskipun struktur DOM merupakan konsep dasar dalam pengembangan web, representasinya dalam bentuk kode HTML sering kali sulit dipahami secara langsung, terutama pada halaman web yang memiliki banyak elemen. Oleh karena itu, diperlukan suatu cara yang dapat membantu menyajikan struktur tersebut dalam bentuk yang lebih mudah dipahami.

Salah satu cara yang dapat digunakan adalah dengan memvisualisasikan struktur halaman web dalam bentuk graf. Dalam pendekatan ini, setiap elemen HTML pada satu halaman web direpresentasikan sebagai simpul (*node*) dan hubungan antar elemen direpresentasikan sebagai sisi (*edge*). Graf yang dimaksud pada penelitian ini adalah graf struktur DOM dari satu halaman web, bukan graf tautan antar halaman pada suatu situs. Dengan demikian, struktur DOM dapat dilihat secara lebih jelas dan terstruktur.

Berdasarkan permasalahan tersebut, tugas akhir ini berfokus pada pengembangan perangkat lunak yang mampu mengambil dokumen HTML dari suatu *Uniform Resource Locator* (URL), membangun struktur DOM, dan merepresentasikan struktur DOM dari halaman tersebut dalam bentuk graf yang dapat divisualisasikan. Perangkat lunak ini diharapkan dapat membantu pengguna dalam memahami struktur halaman web secara lebih mudah.

1.2 Rumusan Masalah

Rumusan masalah yang akan dibahas pada tugas akhir ini adalah:

1. Bagaimana cara mengambil dokumen HTML dari suatu situs web berdasarkan URL serta membangun struktur DOM?
2. Bagaimana cara merepresentasikan struktur DOM ke dalam bentuk graf yang terdiri dari (*node*) dan (*edge*) melalui proses traversal yang sistematis?
3. Bagaimana merancang perangkat lunak yang mampu menghasilkan visualisasi graf dari struktur DOM satu halaman web secara otomatis dan mudah dipahami pengguna?

1.3 Tujuan

Tujuan yang ingin dicapai skripsi ini adalah sebagai berikut:

1. Mengambil dokumen HTML dari suatu situs web berdasarkan URL serta membangun struktur DOM.
2. Merepresentasikan struktur DOM ke dalam bentuk graf yang terdiri dari (*node*) dan (*edge*) melalui proses traversal yang sistematis.
3. Merancang perangkat lunak yang mampu menghasilkan visualisasi graf dari struktur DOM satu halaman web secara otomatis dan mudah dipahami pengguna.

1.4 Batasan Masalah

Pengerjaan tugas akhir ini memiliki batasan sebagai berikut:

- Perangkat lunak dikembangkan menggunakan bahasa pemrograman Java.
- Sistem memproses satu halaman web berdasarkan URL yang dimasukkan oleh pengguna dan tidak melakukan *web crawling* ke halaman lain.
- Sistem menganalisis dokumen HTML yang diperoleh dari URL dan tidak mengeksekusi JavaScript sisi klien.
- Elemen yang dipetakan ke dalam graf adalah elemen HTML pada struktur DOM, sedangkan teks, atribut, dan komentar tidak dijadikan *node* utama.
- Hasil visualisasi graf dibuat menggunakan format DOT dan Graphviz.
- Halaman web yang diproses adalah halaman yang dapat diakses secara publik tanpa autentikasi khusus.

1.5 Metodologi

Metodologi pengerjaan tugas akhir ini adalah sebagai berikut:

1. Melakukan studi literatur mengenai HTML, DOM, struktur data graf, Graphviz, dan metode traversal.
2. Melakukan analisis kebutuhan serta perancangan arsitektur perangkat lunak yang akan dikembangkan.
3. Mengimplementasikan sistem untuk mengambil dokumen HTML, membangun struktur DOM, serta merepresentasikannya dalam bentuk graf.
4. Melakukan pengujian menggunakan beberapa halaman web dengan struktur yang berbeda serta mengevaluasi kesesuaian daftar *node*, daftar *edge*, dan hasil visualisasi graf.
5. Melakukan analisis terhadap hasil visualisasi berdasarkan keterbacaan graf, kesesuaian relasi *parent-child*, dan keberhasilan pembentukan berkas DOT.
6. Menyusun dokumentasi dan laporan tugas akhir.

1.6 Sistematika Pembahasan

Sistematika pembahasan skripsi ini adalah sebagai berikut:

- Bab 1: Pendahuluan Membahas latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan.
- Bab 2: Landasan Teori Membahas dasar dan hierarki HTML, struktur dan jenis *node* DOM, graf berarah, format DOT, dan Graphviz.
- Bab 3: Analisis Membahas analisis masalah, kebutuhan sistem, sistem usulan, model representasi graf, serta kriteria evaluasi sistem.
- Bab 4: Perancangan
- Bab 5: Implementasi dan Pengujian
- Bab 6: Kesimpulan dan Saran

BAB 2

LANDASAN TEORI

2.1 Hypertext Markup Language (HTML)

2.1.1 Pengertian HTML

Hypertext Markup Language (HTML) merupakan bahasa *markup* yang digunakan untuk menyusun struktur dasar halaman web. HTML tidak termasuk bahasa pemrograman karena tidak memiliki mekanisme logika seperti percabangan, perulangan, atau pengolahan data secara langsung, melainkan digunakan untuk mendeskripsikan struktur dan makna konten yang akan ditampilkan pada *browser* [2]. HTML memungkinkan pengembang mengatur berbagai jenis konten, seperti teks, gambar, tautan, tabel, formulir, serta elemen multimedia lainnya dalam sebuah dokumen web.

HTML berfungsi sebagai kerangka utama halaman web yang menentukan bagaimana setiap bagian konten disusun sehingga dapat ditampilkan secara terstruktur oleh *browser* [2]. Dalam pengembangan web, HTML umumnya digunakan bersama dengan CSS dan JavaScript. HTML berperan dalam membentuk struktur konten, CSS digunakan untuk mengatur tampilan atau gaya visual, sedangkan JavaScript digunakan untuk menambahkan interaktivitas pada halaman web.

HTML bekerja dengan konsep *tag-based structure*, yaitu setiap elemen ditandai dengan *tag* tertentu. Tag tersebut memberikan informasi kepada *browser* mengenai jenis dan fungsi konten, misalnya paragraf, judul, gambar, tautan, atau daftar. Melalui struktur ini, *browser* dapat memahami peran masing-masing elemen, menyusun hubungan antar elemen, dan menampilkannya dengan benar kepada pengguna.

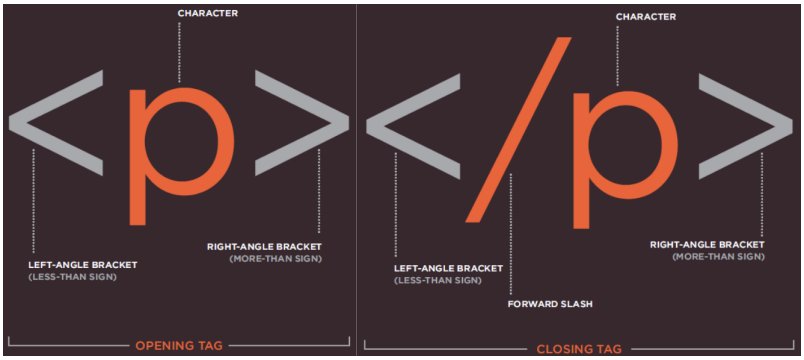
2.1.2 Elemen dan Tag HTML

Elemen HTML merupakan komponen dasar dalam penyusunan halaman web. Setiap elemen digunakan untuk merepresentasikan bagian tertentu dari konten, seperti paragraf, judul, gambar, tautan, tabel, daftar, maupun formulir. Secara umum, sebuah elemen HTML terdiri dari *tag* pembuka, isi elemen, dan *tag* penutup. Sebagai contoh, elemen paragraf dituliskan dengan `<p>` sebagai *tag* pembuka, isi teks sebagai konten, dan `</p>` sebagai *tag* penutup seperti pada gambar 2.1.

Tag berfungsi sebagai penanda yang memberi tahu *browser* mengenai jenis elemen yang sedang digunakan. Tag pembuka menunjukkan awal suatu elemen, sedangkan *tag* penutup menunjukkan akhir elemen tersebut. Namun, tidak semua elemen HTML memiliki *tag* penutup. Beberapa elemen, seperti `<img>`, `<br>`, dan `<hr>`, termasuk elemen kosong karena tidak memiliki isi di antara *tag* pembuka dan *tag* penutup.

Selain *tag*, elemen HTML juga dapat memiliki atribut yang memberikan informasi tambahan terhadap elemen tersebut. Atribut dituliskan di dalam *tag* pembuka dan umumnya terdiri dari nama atribut serta nilai atribut. Sebagai contoh, atribut *href* pada elemen `<a>` digunakan untuk menentukan alamat tautan, sedangkan atribut *src* pada elemen `<img>` digunakan untuk menentukan lokasi berkas gambar. Atribut lain seperti *id* dan *class* sering digunakan untuk memberi identitas atau pengelompokan elemen agar dapat diatur lebih lanjut menggunakan CSS atau diproses melalui JavaScript.

Struktur elemen yang saling bersarang (*nested elements*) memungkinkan terbentuknya hubungan hierarkis antar elemen. Dalam struktur ini, suatu elemen dapat menjadi induk bagi elemen lain yang berada di dalamnya. Misalnya, elemen `<body>` dapat memuat elemen `<div>`, kemudian elemen `<div>` tersebut dapat memuat elemen `<p>`, `<img>`, atau `<a>`. Hubungan bertingkat ini menjadi dasar penting dalam pembentukan struktur DOM karena *browser* akan membaca dokumen HTML sebagai susunan *node* yang saling terhubung.



Gambar 2.1: Contoh *tag* pembuka dan penutup dokumen HTML

2.1.3 Struktur dasar HTML

Dokumen HTML memiliki struktur standar yang terdiri dari beberapa bagian utama, yaitu `<html>`, `<head>`, dan `<body>`. Pada kode 2.1, struktur tersebut diawali dengan deklarasi `<!DOCTYPE html>` yang berfungsi untuk memberi tahu *browser* bahwa dokumen menggunakan standar HTML modern. Setelah deklarasi tersebut, elemen `<html>` digunakan sebagai *root* dari seluruh dokumen HTML dan menjadi wadah bagi elemen `<head>` serta `<body>`.

Bagian `<head>` pada Kode 2.1 berisi informasi yang tidak ditampilkan secara langsung pada halaman, tetapi dibutuhkan oleh *browser* untuk memahami dokumen. Elemen `meta` dengan atribut `charset` digunakan untuk menentukan pengkodean karakter. Elemen `meta` dengan atribut `name="viewport"` digunakan untuk mengatur tampilan halaman pada perangkat yang berbeda, sedangkan elemen `<title>` digunakan untuk menentukan judul dokumen.

Bagian `<body>` pada Kode 2.1 berisi konten utama yang akan ditampilkan kepada pengguna. Di dalamnya terdapat elemen `<h1>` sebagai judul utama, elemen `<h2>` sebagai subjudul, serta elemen `<p>` sebagai paragraf. Susunan tersebut menunjukkan bahwa isi halaman web ditempatkan di dalam `<body>` dan disusun berdasarkan fungsi masing-masing elemen.

Dengan demikian, Kode 2.1 memperlihatkan bahwa dokumen HTML memiliki susunan yang hierarkis. Elemen `<html>` menjadi elemen induk, sedangkan `<head>` dan `<body>` menjadi elemen anak. Di dalam `<body>`, *heading* dan paragraf menjadi bagian dari konten halaman. Susunan yang

teratur ini memudahkan *browser* membaca dokumen, menampilkan halaman secara benar, dan membentuk DOM karena setiap elemen memiliki posisi serta hubungan yang jelas di dalam hierarki dokumen.

Kode 2.1: Contoh struktur dasar dokumen HTML yang terdiri dari deklarasi dokumen; elemen html; bagian head; yang berisi metadata dan title serta bagian body berisi heading dan paragraf sebagai konten utama halaman

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
    <body>
        <h1>This is the Main Heading</h1>
        <p>This text might be an introduction to the rest of the page, And if the page is a long one it might be split up into
            several sub-headings.</p>
            <h2>This is a Sub-Heading</h2>
            <p>Many long articles have sub-heading so to help you follow the structure of what is being written. There may
                even be sub-sub-headings (or lower-level headings).</p>
            <h2>Another Sub-Heading</h2>
            <p>Here you can see another sub-heading</p>
    </body>
</html>
```

2.1.4 Hierarki elemen HTML

Struktur HTML bersifat hierarkis, yang berarti elemen dapat berada di dalam elemen lain. Hubungan antar elemen ini membentuk struktur bertingkat yang menyerupai pohon, di mana setiap elemen memiliki posisi tertentu dalam dokumen. Dalam struktur tersebut, elemen yang memuat elemen lain disebut sebagai *parent*, sedangkan elemen yang berada di dalamnya disebut sebagai *child*. Elemen-elemen yang berada pada tingkat yang sama dan memiliki *parent* yang sama disebut sebagai *sibling*.

Sebagai contoh, elemen `<body>` dapat berperan sebagai *parent* bagi elemen `<div>`. Selanjutnya, elemen `<div>` dapat berisi elemen `<p>`, `<img>`, atau `<a>` sebagai *child*. Jika di dalam satu elemen `<div>` terdapat beberapa elemen `<p>`, maka elemen-elemen tersebut dapat disebut sebagai *sibling* karena berada pada tingkat hierarki yang sama. Hubungan seperti ini menunjukkan bahwa dokumen HTML tidak hanya tersusun secara berurutan, tetapi juga memiliki hubungan induk-anak yang jelas antar elemen.

Hierarki ini penting karena menentukan bagaimana elemen saling berhubungan dan bagaimana struktur tersebut akan diproses menjadi DOM. Ketika *browser* membaca dokumen HTML, setiap elemen akan direpresentasikan sebagai *node* dalam struktur DOM. Hubungan *parent*, *child*, dan *sibling* pada HTML akan menjadi dasar pembentukan hubungan antar *node* di dalam DOM. Oleh karena itu, susunan elemen yang benar akan memudahkan *browser* dalam memahami struktur dokumen dan menampilkan halaman web secara sesuai.

Semakin kompleks halaman web, maka semakin dalam dan luas struktur hierarki yang terbentuk. Halaman yang memiliki banyak bagian, seperti header, navigasi, konten utama, sidebar, formulir, dan footer, akan menghasilkan struktur HTML yang lebih bercabang. Struktur hierarkis tersebut juga memengaruhi proses penelusuran elemen, baik dalam manipulasi DOM menggunakan JavaScript maupun dalam analisis struktur halaman menggunakan pendekatan *traversal* seperti *Depth First Search* (DFS).

2.2 Document Object Model (DOM)

2.2.1 Pengertian DOM

Document Object Model (DOM) adalah representasi dari dokumen HTML dalam bentuk struktur objek yang dapat diakses dan dimanipulasi oleh program. DOM memungkinkan setiap elemen dalam halaman web diperlakukan sebagai objek yang memiliki properti dan metode [1].

Menurut *World Wide Web Consortium*, DOM menyediakan antarmuka standar untuk mengakses dan memodifikasi dokumen HTML. Dengan adanya DOM, halaman web tidak hanya menjadi tampilan statis, tetapi juga dapat diproses dan dimodifikasi secara dinamis.

2.2.2 Struktur DOM sebagai Tree

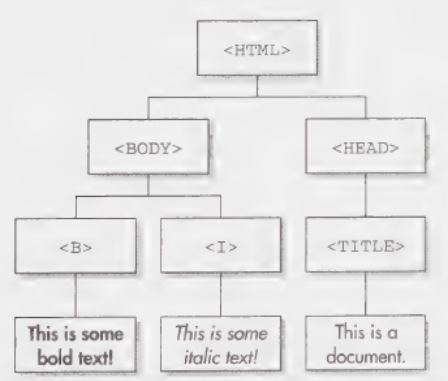
Struktur DOM berbentuk pohon (*tree structure*) karena dokumen HTML disusun dari elemen-elemen yang saling berhubungan secara hierarkis. Dalam DOM, setiap bagian dokumen direpresentasikan sebagai *node*, seperti *node* dokumen, *node* elemen, *node* atribut, dan *node* teks. Menurut Marini, DOM memodelkan dokumen sebagai kumpulan objek yang tersusun dalam struktur pohon sehingga setiap objek dapat diakses, ditelusuri, dan dimodifikasi melalui antarmuka pemrograman [1].

Pada struktur tersebut, *node* paling atas disebut *root node*. Untuk dokumen HTML, elemen `<html>` umumnya menjadi akar utama yang menaungi elemen `<head>` dan `<body>`. Elemen `<head>` dapat memiliki anak seperti `<title>` atau `<meta>`, sedangkan elemen `<body>` dapat memiliki anak seperti `<div>`, `<h1>`, `<p>`, atau elemen lain yang membentuk isi halaman. Hubungan bertingkat ini membuat DOM dapat dipahami sebagai susunan induk dan anak antar elemen, sebagaimana ditunjukkan pada Gambar 2.2.

Setiap *node* dalam DOM dapat memiliki beberapa jenis relasi, yaitu:

- Parent (induk), yaitu *node* yang menaungi *node* lain di bawahnya.
- Child (anak), yaitu *node* yang berada langsung di dalam *node* induk.
- Sibling (saudara), yaitu *node* yang berada pada tingkat yang sama dan memiliki *parent* yang sama.

Relasi *parent*, *child*, dan *sibling* memungkinkan program menelusuri dokumen dari satu *node* ke *node* lain. Penelusuran dapat dilakukan dari *root* menuju *node* turunan, dari suatu *node* menuju *parent*, atau antar *node* yang berada pada level yang sama. Dengan bentuk pohon tersebut, struktur DOM dapat dianalisis secara sistematis menggunakan metode *traversal*, misalnya *Depth First Search* (DFS), untuk membaca seluruh elemen dari akar hingga *node-node* turunannya.



Gambar 2.2: Contoh DOM berbentuk *tree* yang berisi Body dan Head, di dalam Body terdapat elemen *bold* dan *italic*, di dalam Head terdapat elemen judul

2.2.3 Jenis *node* pada DOM

DOM tidak hanya terdiri dari elemen HTML, tetapi juga tersusun dari beberapa jenis *node* yang merepresentasikan bagian berbeda dari dokumen. DOM memandang dokumen sebagai kumpulan objek, sehingga setiap bagian dokumen dapat diperlakukan sebagai *node* yang memiliki jenis, nilai, serta hubungan tertentu dengan *node* lain [1]. Perbedaan jenis *node* ini penting karena program perlu mengetahui apakah suatu *node* merupakan elemen, teks, atribut, atau bagian dokumen lainnya sebelum dilakukan proses pembacaan maupun manipulasi.

Beberapa jenis *node* yang umum terdapat pada DOM antara lain:

- *Document node*, yaitu *node* utama yang merepresentasikan keseluruhan dokumen HTML. Node ini menjadi titik awal untuk mengakses struktur DOM.
- *Element node*, yaitu *node* yang merepresentasikan *tag* HTML seperti `<html>`, `<head>`, `<body>`, `<div>`, `<p>`, atau `<a>`. *Node* jenis ini biasanya memiliki relasi *parent* dan *child* dengan elemen lain.
- *Text node*, yaitu *node* yang merepresentasikan isi teks yang berada di dalam suatu elemen. Misalnya, teks dalam elemen `<p>` akan dibaca sebagai *node* teks.
- *Attribute node*, yaitu *node* yang merepresentasikan atribut pada elemen HTML, seperti *id*, *class*, *href*, atau *src*. Atribut memberikan informasi tambahan terhadap elemen yang memilikinya.
- *Comment node*, yaitu *node* yang merepresentasikan komentar di dalam dokumen HTML. Node ini tidak ditampilkan sebagai konten utama halaman, tetapi tetap dapat menjadi bagian dari struktur dokumen.

Keberadaan berbagai jenis *node* tersebut membuat DOM lebih kaya dibandingkan HTML mentah karena DOM tidak hanya menyimpan *tag*, tetapi juga menyimpan teks, atribut, komentar, serta relasi antarbagian dokumen. Dengan demikian, analisis DOM dapat memberikan informasi yang lebih lengkap mengenai struktur halaman web. Pemahaman terhadap jenis *node* diperlukan agar elemen-elemen yang relevan dapat dipilih dan dipetakan ke dalam representasi graf secara tepat.

2.3 Jsoup

2.3.1 Pengertian Jsoup

Jsoup merupakan *library* Java yang digunakan untuk membaca, memproses, mengekstraksi, dan memanipulasi dokumen HTML. *Library* ini menyediakan antarmuka pemrograman yang memudahkan pengembang dalam mengambil data dari dokumen HTML, baik yang berasal dari berkas lokal, *string* HTML, maupun halaman web yang diakses melalui *URL* [3]. Dengan menggunakan Jsoup, struktur HTML dapat diproses menjadi bentuk dokumen yang lebih mudah ditelusuri dan dianalisis oleh program.

Jsoup bekerja dengan cara mem-parsing kode HTML dan membentuk representasi dokumen yang menyerupai struktur DOM. Setelah dokumen diproses, elemen-elemen HTML dapat diakses berdasarkan *tag*, atribut, teks, maupun relasi antar elemen. Hal ini membuat Jsoup banyak digunakan dalam pengolahan dokumen web, seperti ekstraksi informasi, pembersihan data HTML, analisis struktur halaman, dan pengambilan elemen tertentu dari sebuah halaman web.

Kode 2.2: Potongan kode dengan fungsi mengambil *homepage* Wikipedia lalu melakukan *parse* menjadi DOM dan memilih judul berita dari bagian "Berita" ke dalam daftar Elemen

```
    Document doc = Jsoup.connect("https://en.wikipedia.org/").get();
    log(doc.title());
    Elements newsHeadlines = doc.select("#mp-itn b a");
    for (Element headline : newsHeadlines) {
        log("%s\n\t%s",
            headline.attr("title"), headline.absUrl("href"));
}
```

Penjelasan setiap baris pada Kode 2.2 adalah sebagai berikut [3]:

1. Baris pertama mengambil dan melakukan *parsing* HTML halaman utama Wikipedia, kemudian menyimpan hasilnya sebagai objek `Document` pada variabel `doc`.
2. Baris kedua mengambil teks judul dokumen melalui `doc.title()` dan meneruskannya ke fungsi `log` untuk ditampilkan.
3. Baris ketiga memilih elemen `a` di dalam elemen `b` yang berada pada bagian ber-ID `mp-itn`, lalu menyimpan kumpulan tautan tersebut dalam `newsHeadlines`.
4. Baris keempat memulai perulangan untuk mengakses setiap elemen dalam `newsHeadlines` melalui variabel `headline`.
5. Baris kelima memulai pemanggilan `log` dengan format dua nilai teks: `%s` sebagai tempat nilai, `\n` sebagai baris baru, dan `\t` sebagai tabulasi.
6. Baris keenam melengkapi pemanggilan tersebut dengan nilai atribut `title` dan URL absolut dari atribut `href` pada tautan yang sedang diproses.
7. Baris ketujuh menutup blok perulangan.

Fungsi `log` pada contoh tersebut merupakan fungsi bantu untuk menampilkan keluaran, bukan metode bawaan Jsoup, sehingga perlu didefinisikan dalam program yang menjalankan contoh ini.

2.3.2 Kelas `Jsoup`

Kelas `Jsoup` merupakan titik masuk utama untuk menggunakan pustaka Jsoup. Berikut adalah beberapa metode yang tersedia pada kelas ini:

- *connect:* Metode ini digunakan untuk membuat koneksi HTTP menuju URL situs web tujuan.

- *parse:* Metode ini digunakan untuk melakukan *parsing* pada `String` HTML menjadi objek `Document`.
- *parseBodyFragment:* Metode ini digunakan untuk melakukan *parsing* pada potongan HTML parsial dan menghasilkan objek `Document`.

2.3.3 Kelas `Document`

Kelas `Document` merupakan representasi dari keseluruhan dokumen HTML setelah proses *parsing*. Kelas ini berfungsi sebagai akar struktur DOM dan menyediakan metode untuk membaca, menelusuri, atau memodifikasi isi dokumen. Berikut adalah beberapa metode yang tersedia pada kelas ini:

- *title:* Metode ini digunakan untuk mendapatkan isi teks dari elemen `<title>` dalam bentuk `String`. Jika elemen `<title>` diperlukan sebagai objek elemen, elemen tersebut dapat diperoleh menggunakan metode seleksi seperti `selectFirst("title")`.
- *body:* Metode ini digunakan untuk mendapatkan elemen `<body>` dari dokumen.
- *select:* Metode ini digunakan untuk mencari elemen dalam dokumen berdasarkan CSS *selector*.
- *baseUri:* Metode ini digunakan untuk mendapatkan URL dasar dari dokumen, jika sumbernya diambil melalui URL.

2.3.4 Kelas Element

Kelas `Element` merupakan representasi satu elemen HTML dalam struktur DOM. Kelas ini memiliki *tag*, atribut, konten teks, serta daftar elemen anak. Berikut adalah beberapa metode yang tersedia pada kelas ini:

- *attr:* Metode ini digunakan untuk mengambil nilai suatu atribut.
- *text:* Metode ini digunakan untuk mendapatkan teks dalam elemen beserta elemen turunannya.
- *absUrl:* Metode ini digunakan untuk mendapatkan URL absolut dari sebuah atribut, seperti `href` atau `src`, berdasarkan URI dasar dokumen.
- *html:* Metode ini digunakan untuk memperoleh markup HTML bagian dalam elemen.
- *children:* Metode ini digunakan untuk mendapatkan daftar elemen anak dari sebuah elemen tertentu.

2.3.5 Penggunaan Jsoup dalam Pemrosesan DOM

Dalam konteks pemrosesan DOM, Jsoup berperan sebagai alat bantu untuk mengubah dokumen HTML mentah menjadi objek dokumen yang dapat ditelusuri secara sistematis. Setiap elemen HTML yang telah diproses dapat diakses sebagai *node* yang memiliki hubungan dengan elemen lain, seperti *parent*, *child*, dan *sibling*. Relasi tersebut memungkinkan program membaca struktur halaman secara hierarkis tanpa harus memproses teks HTML secara manual.

Pada penelitian ini, Jsoup digunakan sebagai tahap awal untuk memperoleh struktur DOM dari dokumen HTML. Elemen-elemen yang dihasilkan oleh Jsoup dapat dipetakan menjadi *node*, sedangkan hubungan antar elemen seperti *parent* dan *child* dapat dipetakan menjadi *edge*. Contoh penggunaan Jsoup untuk melakukan *parsing* HTML dan mengambil elemen DOM ditunjukkan pada Kode 2.3. Dengan demikian, Jsoup menjadi penghubung antara dokumen HTML mentah dan representasi graf yang dapat dianalisis serta divisualisasikan.

Kode 2.3: Menggunakan metode DOM setelah *parsing* HTML ke dokumen

```
1
2 1     String url = "https://en.wikipedia.org/";
3 2     Document doc = Jsoup.connect(url).get();
4 3
5 4     Element content = doc.getElementById("content");
6 5     Elements links = content.getElementsByTag("a");
7 6     for (Element link : links) {
8 7         String linkHref = link.attr("href");
9 8         String linkText = link.text();
10 9    }
11
```

Penjelasan setiap baris pada Kode 2.3, adalah sebagai berikut [3]:

1. Baris pertama menyimpan alamat halaman Wikipedia dalam variabel `url`.
2. Baris kedua mengambil dan melakukan *parsing* HTML dari alamat tersebut, lalu menyimpan hasilnya sebagai objek `Document` pada variabel `doc`.
3. Baris ketiga mencari elemen ber-ID `content` dalam dokumen dan menyimpannya dalam variabel `content`.
4. Baris keempat mengambil seluruh elemen bertag `a` di dalam elemen `content`, termasuk turunannya, lalu menyimpannya dalam kumpulan `links`.
5. Baris kelima memulai perulangan untuk mengakses setiap elemen tautan dalam `links` melalui variabel `link`.
6. Baris keenam membaca nilai atribut `href` dan menyimpannya dalam `linkHref`. Nilai ini dapat berupa URL relatif maupun absolut.
7. Baris ketujuh mengambil isi teks tautan beserta teks elemen turunannya tanpa menyertakan *tag* HTML, lalu menyimpannya dalam `linkText`.
8. Baris kedelapan menutup blok perulangan.

Contoh tersebut mengasumsikan bahwa elemen ber-ID `content` tersedia. Jika elemen tidak ditemukan, variabel `content` bernilai `null` sehingga perlu diperiksa sebelum metode `getElementsByTag` dipanggil.

2.4 Struktur Data Tree

2.4.1 Pengertian Tree

Tree adalah struktur data hierarkis yang terdiri dari sekumpulan *node* dan hubungan antar *node* [4]. Node paling atas dalam *tree* disebut *root*, sedangkan *node* yang berada di bawah *node* lain disebut *child*. Node yang memiliki *child* disebut *parent*, sementara *node* yang tidak memiliki *child* disebut leaf. Struktur *tree* digunakan untuk merepresentasikan hubungan bertingkat, yaitu satu *node* dapat memiliki beberapa *node* turunan.

Secara konsep, *tree* merupakan bentuk khusus dari graf yang tidak memiliki siklus dan memiliki hubungan hierarkis yang jelas. Setiap *node* dalam *tree*, kecuali *root*, memiliki satu *parent*. Karakteristik ini membuat *tree* cocok digunakan untuk memodelkan struktur yang tersusun secara bertingkat, seperti struktur folder, organisasi data, ekspresi matematika, dan dokumen HTML.

2.4.2 Tree pada Struktur DOM

DOM memiliki karakteristik yang menyerupai *tree* karena setiap elemen dalam dokumen HTML tersusun secara hierarkis. Elemen `<html>` biasanya menjadi *root*, sedangkan elemen `<head>` dan

`<body>` menjadi *child* dari elemen tersebut. Selanjutnya, elemen-elemen lain seperti `<div>`, `<p>`, `<img>`, dan `<a>` dapat menjadi turunan dari elemen `<body>` atau elemen lain di dalamnya.

Pemahaman mengenai *tree* penting dalam penelitian ini karena struktur DOM yang berbentuk *tree* dapat ditelusuri menggunakan algoritma *traversal*. Salah satu algoritma yang sesuai untuk struktur seperti ini adalah DFS, karena DFS menelusuri *node* secara mendalam dari *root* menuju leaf sebelum kembali ke cabang lain. Dengan demikian, konsep *tree* menjadi dasar untuk memahami hubungan antara DOM, graf, dan proses *traversal*.

2.5 Struktur Data Graf

2.5.1 Pengertian Graf

Graf adalah struktur data yang terdiri dari simpul (*node* atau *vertex*) dan sisi (*edge*) yang menghubungkan simpul-simpul tersebut. Dalam kajian algoritma, graf umumnya dinyatakan sebagai pasangan himpunan $G = (V, E)$, dengan V sebagai himpunan simpul dan E sebagai himpunan sisi yang merepresentasikan hubungan antar simpul [4]. Setiap simpul dapat menggambarkan suatu objek, sedangkan sisi menunjukkan adanya relasi atau keterhubungan antara dua objek.

Menurut Cormen, graf banyak digunakan untuk memodelkan berbagai jenis hubungan dalam komputasi karena bentuknya fleksibel dan dapat merepresentasikan struktur yang sederhana maupun kompleks [4]. Sebagai contoh, simpul dapat merepresentasikan kota, halaman web, pengguna, proses, atau elemen dalam dokumen, sedangkan sisi dapat merepresentasikan jalan, tautan, hubungan sosial, komunikasi antar proses, atau hubungan induk-anak antar elemen.

Graf dapat dibedakan berdasarkan karakteristik sisinya. Pada graf tidak berarah, hubungan antar simpul tidak memiliki arah tertentu sehingga relasi antara dua simpul dianggap berlaku dua arah. Sebaliknya, pada graf berarah, setiap sisi memiliki arah dari satu simpul ke simpul lainnya. Selain itu, graf juga dapat memiliki bobot pada sisi untuk menyatakan nilai tertentu, seperti jarak, biaya, waktu, atau tingkat hubungan antar simpul.

Dalam konteks penelitian ini, konsep graf digunakan untuk merepresentasikan struktur DOM dari dokumen HTML. Setiap elemen HTML dapat dipandang sebagai simpul, sedangkan hubungan antara elemen induk dan elemen anak dapat dipandang sebagai sisi. Dengan pemodelan tersebut, struktur halaman web dapat dianalisis menggunakan konsep teori graf, seperti hubungan antar *node*, arah hubungan, dan proses *traversal*.

2.5.2 Representasi Graf

Representasi graf adalah cara menyimpan struktur graf agar dapat diproses oleh komputer. Dalam implementasi algoritma, graf tidak cukup hanya didefinisikan sebagai himpunan simpul dan sisi, tetapi perlu direpresentasikan dalam bentuk data tertentu. Dua bentuk representasi graf yang umum digunakan adalah adjacency matrix dan adjacency list [4].

Adjacency matrix merepresentasikan graf dalam bentuk matriks dua dimensi. Setiap baris dan kolom mewakili simpul, sedangkan nilai pada sel matriks menunjukkan ada atau tidaknya sisi antara dua simpul. Representasi ini mudah dipahami dan efisien untuk memeriksa hubungan langsung antar simpul, tetapi dapat menggunakan ruang penyimpanan yang besar jika jumlah simpul banyak dan jumlah sisi sedikit.

Adjacency list merepresentasikan graf dengan menyimpan daftar tetangga untuk setiap simpul. Setiap simpul memiliki daftar *node* lain yang terhubung dengannya. Representasi ini lebih hemat ruang untuk graf yang memiliki jumlah sisi relatif sedikit dibandingkan jumlah kemungkinan sisi. Dalam konteks DOM, adjacency list sesuai digunakan karena setiap elemen biasanya hanya memiliki hubungan langsung dengan sejumlah *child* tertentu, bukan dengan seluruh elemen dalam dokumen.

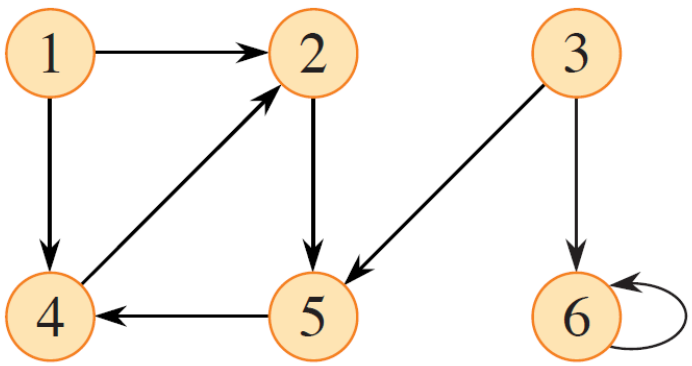
2.5.3 Graf Berarah

Graf berarah (*directed graph* atau *digraph*) adalah graf yang setiap sisinya memiliki arah tertentu. Pada graf berarah, sisi tidak hanya menunjukkan bahwa dua simpul saling terhubung, tetapi juga menunjukkan arah hubungan dari satu simpul menuju simpul lainnya. Secara formal, sisi pada graf berarah dapat dinyatakan sebagai pasangan terurut (u, v), yang berarti sisi tersebut berawal dari simpul u dan menuju simpul v [4].

Berbeda dengan graf tidak berarah, hubungan pada graf berarah tidak selalu berlaku dua arah. Jika terdapat sisi dari simpul u ke simpul v, maka hal tersebut tidak secara otomatis berarti terdapat sisi dari simpul v ke simpul u. Oleh karena itu, arah sisi menjadi informasi penting dalam memahami pola hubungan antar simpul. Konsep ini banyak digunakan untuk memodelkan hubungan yang memiliki alur, urutan, atau ketergantungan, seperti tautan antar halaman web, aliran data, proses kerja, maupun struktur hierarkis.

Dalam konteks DOM, hubungan antara *parent* dan *child* memiliki arah yang jelas, yaitu dari *node* induk menuju *node* anak. Misalnya, elemen `<html>` memiliki hubungan ke elemen `<head>` dan `<body>`, kemudian elemen `<body>` dapat memiliki hubungan ke elemen `<div>`, `<p>`, atau elemen lainnya. Arah hubungan tersebut menunjukkan bahwa suatu elemen berada di bawah elemen lain dalam struktur dokumen.

Pemodelan DOM sebagai graf berarah membantu menunjukkan alur hubungan antar elemen secara lebih terstruktur. Dengan menggunakan arah sisi, proses penelusuran dapat dilakukan dari *root node* menuju *node-node* turunannya. Hal ini penting dalam analisis struktur halaman web karena urutan dan arah hubungan antar elemen memengaruhi cara DOM dibaca, divisualisasikan, dan ditelusuri menggunakan algoritma *traversal* seperti *Depth First Search*. Contoh bentuk graf berarah sederhana yang terdiri dari *node* dan *edge* ditunjukkan pada Gambar 2.3.



Gambar 2.3: Contoh graf berarah sederhana (*node* dan *edge*)

2.5.4 Representasi DOM sebagai graf

Struktur DOM dapat direpresentasikan sebagai graf dengan cara memetakan setiap elemen HTML sebagai *node* dan hubungan *parent-child* sebagai *edge*. Dalam representasi ini, elemen *root* seperti `<html>` menjadi *node* awal, sedangkan elemen-elemen turunannya seperti `<head>`, `<body>`, `<div>`, `<p>`, dan elemen lain menjadi *node* yang terhubung melalui *edge*.

Karena hubungan antar elemen dalam DOM memiliki arah dari *parent* menuju *child*, struktur DOM lebih sesuai dimodelkan sebagai graf berarah. Setiap *edge* menunjukkan hubungan hierarkis antara dua elemen. Misalnya, hubungan dari `<body>` ke `<div>` menunjukkan bahwa elemen `<div>` berada di dalam elemen `<body>`. Dengan pemetaan ini, struktur HTML yang semula berbentuk dokumen teks dapat dianalisis sebagai struktur *node* dan *edge*.

Representasi DOM sebagai graf mempermudah proses analisis dan visualisasi struktur halaman web. Melalui pendekatan graf, hubungan antar elemen dapat ditelusuri menggunakan algoritma *traversal* seperti DFS. Selain itu, representasi ini juga mempermudah pembuatan visualisasi menggunakan Graphviz karena struktur DOM dapat diubah ke dalam format DOT dan ditampilkan sebagai diagram graf.

2.5.5 Pengertian Traversal Graf

Traversal graf adalah proses mengunjungi setiap *node* dalam graf secara sistematis berdasarkan aturan tertentu. Proses ini dilakukan untuk menelusuri hubungan antar *node* sehingga seluruh struktur graf dapat diproses tanpa ada *node* yang terlewat. Dalam kajian algoritma, *traversal* menjadi langkah penting karena banyak permasalahan graf membutuhkan proses pemeriksaan *node* dan *edge*, misalnya untuk mencari jalur, menentukan keterhubungan, menemukan komponen, atau menganalisis struktur hubungan antar objek [4].

Pada proses *traversal*, suatu *node* biasanya dipilih sebagai titik awal, kemudian algoritma akan menelusuri *node-node* lain yang terhubung melalui *edge*. Urutan kunjungan *node* bergantung pada metode *traversal* yang digunakan. Beberapa metode *traversal* yang umum digunakan adalah *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS). BFS menelusuri *node* berdasarkan tingkat kedekatan dari *node* awal, sedangkan DFS menelusuri *node* secara mendalam hingga mencapai cabang terdalam sebelum kembali ke *node* sebelumnya.

Dalam konteks penelitian ini, *traversal* graf digunakan untuk menelusuri struktur DOM yang telah direpresentasikan sebagai graf. Setiap elemen HTML dipandang sebagai *node*, sedangkan hubungan *parent-child* dipandang sebagai *edge*. Dengan *traversal*, struktur halaman web dapat dibaca secara teratur dari *node* awal menuju *node-node* turunannya. Hal ini membantu proses analisis DOM karena setiap elemen dapat diperiksa sesuai urutan hubungan hierarkisnya.

2.5.6 Depth-First Search

Depth-First Search (DFS) adalah metode *traversal* graf yang bekerja dengan menelusuri suatu simpul hingga ke bagian terdalam terlebih dahulu sebelum kembali ke simpul sebelumnya. Sesuai dengan namanya, DFS akan memilih untuk bergerak lebih dalam pada graf selama masih terdapat sisi yang belum dieksplorasi dari simpul yang sedang dikunjungi.

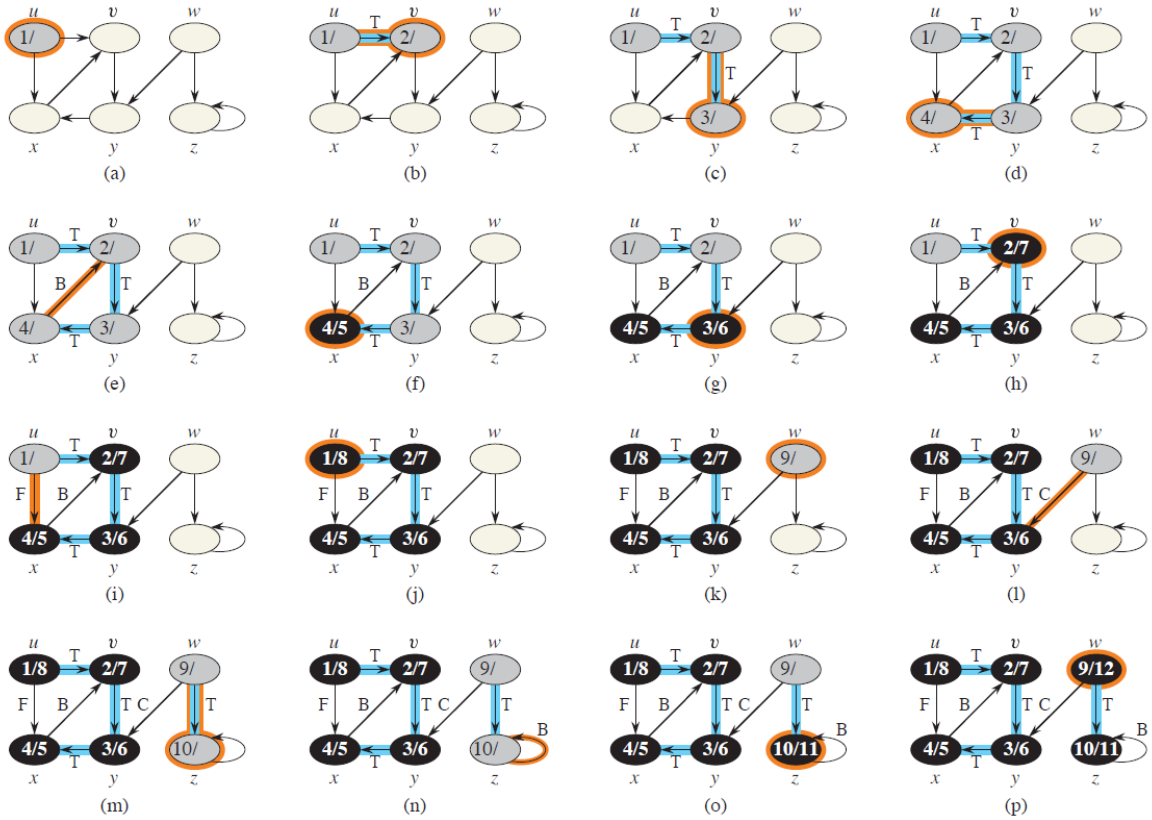
Menurut Cormen, DFS mengeksplorasi sisi-sisi yang keluar dari simpul yang paling baru

ditemukan dan masih memiliki sisi yang belum diperiksa. Jika seluruh sisi dari simpul tersebut telah dieksplorasi, proses pencarian akan melakukan *backtracking*, yaitu kembali ke simpul asal tempat simpul tersebut ditemukan untuk melanjutkan eksplorasi sisi lain yang belum dikunjungi [4]. Proses ini berlanjut hingga seluruh simpul yang dapat dicapai dari simpul awal berhasil ditemukan. Apabila masih terdapat simpul lain yang belum ditemukan, DFS akan memilih simpul tersebut sebagai sumber baru dan mengulangi proses pencarian sampai seluruh simpul dalam graf telah diperiksa.

Dalam prosesnya, DFS juga mencatat hubungan pendahulu (*predecessor*) dari setiap simpul yang ditemukan. Ketika suatu simpul baru ditemukan dari simpul lain, hubungan tersebut dicatat sebagai bagian dari struktur hasil pencarian. Pada graf yang tidak seluruh simpulnya terhubung dari satu sumber, DFS dapat menghasilkan beberapa pohon pencarian, sehingga hasil keseluruhannya disebut sebagai *depth-first forest*. Sisi-sisi yang membentuk hubungan pendahulu tersebut dikenal sebagai *tree edges* [4].

DFS menggunakan penandaan warna untuk menunjukkan status setiap simpul selama proses *traversal*. Simpul yang belum dikunjungi diberi warna putih, simpul yang sedang diproses diberi warna abu-abu, dan simpul yang seluruh sisi keluarnya telah selesai diperiksa diberi warna hitam. Selain itu, DFS juga mencatat dua waktu pada setiap simpul, yaitu waktu penemuan (*discovery time*) ketika simpul pertama kali dikunjungi dan waktu penyelesaian (*finishing time*) ketika seluruh daftar ketetanggaan simpul tersebut telah selesai diperiksa. Informasi ini berguna untuk memahami urutan *traversal* dan struktur hubungan antar simpul dalam graf [4].

Metode ini sangat cocok untuk struktur berbentuk *tree* seperti DOM karena mengikuti pola *parent-child* secara mendalam. Pada konteks DOM, DFS dapat digunakan untuk menelusuri elemen dari *node* induk ke *node* anak secara bertahap hingga mencapai elemen terdalam, kemudian kembali ke *node* sebelumnya untuk melanjutkan penelusuran ke cabang lain. Ilustrasi proses DFS pada graf berarah dapat dilihat pada Gambar 2.4. Oleh karena itu, DFS sering digunakan untuk eksplorasi struktur hierarkis dan graf [4].



Gambar 2.4: Ilustrasi proses DFS pada graf berarah. Sisi-sisi diklasifikasikan sesuai urutan penjelajahannya: sisi pohon diberi *label* T, sisi belakang B, sisi depan F, dan sisi silang C.

2.6 Graphviz

2.6.1 Pengertian Graphviz

Graphviz adalah perangkat lunak visualisasi grafik *open source*. Visualisasi grafik adalah cara untuk merepresentasikan informasi struktural sebagai diagram grafik abstrak dan jaringan. Teknologi ini memiliki aplikasi penting dalam jaringan, bioinformatika, rekayasa perangkat lunak, desain basis data dan web, pembelajaran mesin, dan antarmuka visual untuk domain teknis lainnya [5].

Menurut *Graphviz Project*, Graphviz menyediakan berbagai algoritma *layout* untuk menampilkan graf secara otomatis.

2.6.2 Format DOT

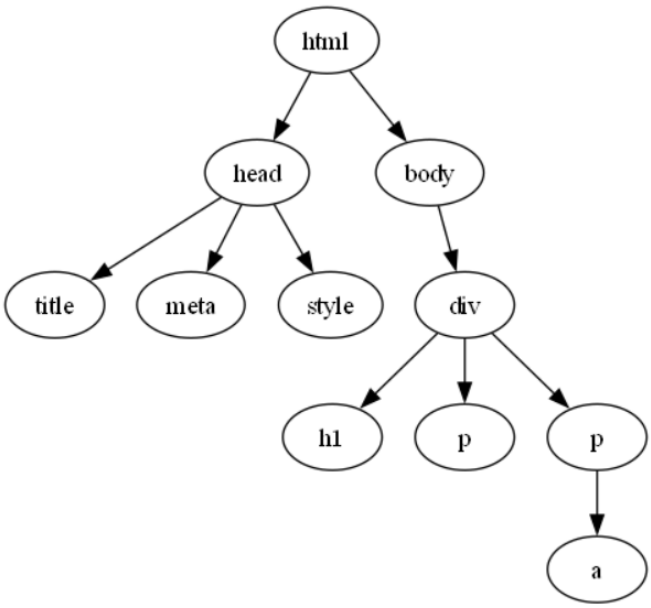
Graphviz menggunakan format DOT untuk mendefinisikan graf. DOT merupakan bahasa deskripsi graf berbasis teks yang digunakan untuk menuliskan struktur *node* dan *edge* secara sederhana. Melalui format ini, pengguna dapat menentukan jenis graf, nama graf, hubungan antar *node*, serta atribut visual seperti *label*, warna, bentuk *node*, dan arah hubungan. Graf tidak berarah pada DOT didefinisikan menggunakan kata kunci *graph* dengan simbol –, sedangkan graf berarah didefinisikan menggunakan kata kunci *digraph* dengan simbol ->.

Format DOT sesuai digunakan untuk merepresentasikan struktur DOM karena hubungan antar elemen HTML bersifat hierarkis dan berarah dari *parent* menuju *child*. Misalnya, hubungan antara

elemen HTML dan *head* dapat dituliskan sebagai `HTML -> head`, yang menunjukkan adanya *edge* dari *node* HTML menuju *node head*. Contoh penulisan struktur DOM dalam format DOT ditunjukkan pada Kode 2.4, sedangkan hasil visualisasinya menggunakan Graphviz ditunjukkan pada Gambar 2.5. Dalam konteks penelitian ini, DOT digunakan sebagai format perantara untuk mengubah struktur DOM menjadi representasi graf yang kemudian dapat divisualisasikan menggunakan Graphviz.

Kode 2.4: Contoh kode .dot

```
digraph DOM {
node [shape=oval];
node1 [label="html"];
node2 [label="head"];
node1 -> node2;
node3 [label="title"];
node2 -> node3;
node4 [label="meta"];
node2 -> node4;
node5 [label="style"];
node2 -> node5;
node6 [label="body"];
node1 -> node6;
node7 [label="div"];
node6 -> node7;
node8 [label="h1"];
node7 -> node8;
node9 [label="p"];
node7 -> node9;
node10 [label="p"];
node7 -> node10;
node11 [label="a"];
node10 -> node11;
}
```



Gambar 2.5: Visualisasi dari Kode 2.4 dengan format DOT dengan menggunakan Graphviz

BAB 3

ANALISIS

3.1 Analisis Masalah

3.1.1 Analisis Struktur HTML

Halaman web tersusun dari elemen-elemen HTML yang memiliki fungsi berbeda, seperti judul, paragraf, tautan, gambar, tabel, dan formulir. Setiap elemen tersebut ditandai menggunakan *tag* dan dapat memiliki atribut sebagai informasi tambahan. Permasalahan muncul ketika struktur HTML yang dianalisis memiliki jumlah elemen yang banyak, susunan yang saling bersarang, serta atribut yang beragam. Kondisi tersebut membuat pembacaan struktur halaman secara manual menjadi kurang efektif karena hubungan antar elemen tidak selalu mudah terlihat dari kode HTML mentah.

Struktur HTML juga memiliki hubungan hierarkis antar elemen. Suatu elemen dapat menjadi induk bagi elemen lain, sedangkan elemen yang berada di dalamnya menjadi anak dari elemen tersebut. Hubungan ini penting karena menentukan bagaimana dokumen HTML dibentuk menjadi DOM. Jika struktur tersebut hanya dilihat dalam bentuk teks, pengguna perlu menelusuri pasangan *tag* pembuka dan penutup secara berulang untuk mengetahui letak suatu elemen, kedalaman elemen, serta hubungan *parent-child* antar elemen.

Berdasarkan permasalahan tersebut, diperlukan proses analisis yang dapat mengubah dokumen HTML menjadi representasi yang lebih mudah dipahami. Elemen HTML perlu dipetakan sebagai *node*, sedangkan hubungan antar elemen perlu dipetakan sebagai *edge*. Dengan pemetaan tersebut, struktur DOM halaman web dapat dianalisis sebagai bentuk graf atau *tree* sehingga hubungan antar elemen dapat terlihat lebih jelas.

Analisis pada penelitian ini berfokus pada bagaimana struktur HTML diproses menjadi DOM, kemudian direpresentasikan dalam bentuk graf berarah. Representasi ini membantu pengguna memahami susunan elemen, melihat hubungan hierarkis antar elemen, serta menelusuri struktur DOM menggunakan pendekatan *traversal*. Dengan demikian, proses analisis tidak hanya melihat isi dokumen HTML, tetapi juga memperhatikan relasi struktural yang terbentuk di dalam halaman web.

3.1.2 Analisis Pembentukan DOM

Dokumen HTML yang diperoleh dari suatu situs web tidak dapat langsung divisualisasikan sebagai graf karena masih berbentuk teks berisi kumpulan *tag*, atribut, dan konten. Agar struktur halaman dapat dianalisis oleh sistem, dokumen HTML perlu diproses terlebih dahulu menjadi DOM. DOM

merepresentasikan dokumen sebagai susunan objek yang memiliki hubungan hierarkis sehingga setiap elemen dapat diakses sebagai bagian dari struktur yang lebih teratur.

Permasalahan dalam pembentukan DOM adalah tidak semua dokumen HTML memiliki susunan yang sederhana dan rapi. Halaman web dapat memiliki elemen yang sangat banyak, elemen kosong, atribut yang beragam, serta elemen yang saling bersarang pada tingkat kedalaman berbeda. Selain itu, halaman web yang diambil dari URL tertentu juga dapat memiliki struktur yang berbeda-beda antara satu situs dengan situs lainnya. Oleh karena itu, sistem membutuhkan proses *parsing* yang mampu membaca dokumen HTML dan membentuk struktur DOM secara konsisten.

Pada sistem yang diusulkan, Jsoup dimanfaatkan untuk mengambil sekaligus mengonversi HTML menjadi struktur DOM. URL yang telah lolos validasi diberikan kepada Jsoup melalui metode `connect(url)`, kemudian metode `get()` digunakan untuk mengirim permintaan, mengambil respons HTML, dan melakukan *parsing*. Hasil proses tersebut berupa objek `Document` yang menjadi akar struktur DOM. Dengan demikian, sistem tidak perlu memisahkan proses pengambilan HTML dan pembentukan DOM secara manual karena keduanya dapat dilakukan melalui pemanggilan `Jsoup.connect(url).get()`.

Objek `Document` selanjutnya digunakan untuk mengakses elemen-elemen HTML secara hierarkis. Sistem memulai penelusuran dari elemen akar, mengambil nama *tag* melalui metode `tagName()`, dan memperoleh elemen anak melalui metode `children()`. Penggunaan `children()` membuat penelusuran hanya memproses elemen HTML, sehingga *text node* dan *comment node* tidak dimasukkan sebagai simpul graf. Untuk setiap elemen anak yang ditemukan, sistem mempertahankan referensi elemen induknya agar relasi *parent-child* dapat dibentuk sebagai *edge*. Apabila proses koneksi atau *parsing* gagal, sistem menangkap kesalahan tersebut dan menampilkan pesan kepada pengguna.

DOM yang dibentuk oleh Jsoup merepresentasikan HTML yang diterima dari respons server. Jsoup tidak mengeksekusi JavaScript pada halaman, sehingga elemen yang baru dibuat atau diubah oleh JavaScript setelah halaman ditampilkan pada peramban tidak termasuk dalam struktur DOM yang diproses oleh sistem.

Dalam penelitian ini, pembentukan DOM menjadi tahap penting karena hasil dari proses tersebut menjadi dasar untuk membangun graf. Apabila elemen dan hubungan antar elemen tidak terbaca dengan benar, maka graf yang dihasilkan juga tidak dapat menggambarkan struktur DOM halaman web secara tepat. Dengan demikian, sistem perlu memastikan bahwa elemen-elemen pada DOM dapat dikenali dan relasi antar elemen dapat dipertahankan sebelum masuk ke tahap pemetaan graf.

3.1.3 Analisis Representasi DOM sebagai Graf

Struktur DOM memiliki karakteristik yang menyerupai *tree* karena elemen-elemen di dalamnya tersusun secara hierarkis dari elemen akar menuju elemen turunan. Meskipun demikian, dalam bentuk DOM biasa, hubungan tersebut belum tentu mudah dipahami oleh pengguna. Oleh karena itu, diperlukan representasi graf untuk menampilkan hubungan antar elemen secara visual.

Pada representasi graf, setiap elemen HTML dipandang sebagai *node*, sedangkan hubungan *parent-child* antar elemen dipandang sebagai *edge*. Arah *edge* menunjukkan hubungan dari elemen induk menuju elemen anak. Dengan cara ini, struktur DOM dapat dianalisis sebagai graf berarah

yang memperlihatkan susunan elemen secara lebih jelas dibandingkan tampilan kode HTML mentah.

Permasalahan yang perlu diperhatikan dalam representasi ini adalah bagaimana sistem menentukan elemen mana saja yang akan dijadikan *node*, bagaimana hubungan antar elemen dibentuk menjadi *edge*, dan bagaimana penamaan *node* dilakukan agar hasil graf tetap mudah dibaca. Jika halaman web memiliki banyak elemen dengan nama *tag* yang sama, sistem perlu tetap membedakan setiap *node* agar hubungan antar elemen tidak tertukar.

3.1.4 Analisis Traversal DOM

Setelah struktur DOM terbentuk, sistem perlu menelusuri seluruh elemen agar setiap elemen dapat dipetakan ke dalam graf. Proses penelusuran ini tidak dapat dilakukan secara acak karena DOM memiliki susunan hierarkis. Jika penelusuran tidak dilakukan secara sistematis, terdapat kemungkinan elemen tertentu terlewat atau hubungan antar elemen tidak terbentuk dengan benar.

Metode *Depth-First Search* (DFS) sesuai digunakan karena pola kerjanya mengikuti struktur hierarkis DOM. DFS menelusuri elemen dari *node* induk menuju *node* anak hingga mencapai elemen terdalam, kemudian kembali ke elemen sebelumnya untuk melanjutkan penelusuran pada cabang lain. Dengan pola tersebut, sistem dapat membaca struktur DOM dari akar hingga seluruh elemen turunannya secara teratur.

Dalam konteks perangkat lunak yang dikembangkan, DFS digunakan untuk membantu proses pembentukan daftar *node* dan *edge*. Setiap kali elemen ditemukan, elemen tersebut dapat dicatat sebagai *node*. Ketika elemen tersebut memiliki hubungan dengan elemen induknya, hubungan tersebut dicatat sebagai *edge*. Dengan demikian, proses *traversal* menjadi dasar dalam menghasilkan representasi graf yang lengkap dan sesuai dengan struktur DOM.

3.1.5 Analisis Kebutuhan Visualisasi

Tujuan utama perangkat lunak ini adalah menghasilkan visualisasi graf dari struktur DOM halaman web. Visualisasi dibutuhkan karena struktur DOM yang kompleks sulit dipahami apabila hanya ditampilkan dalam bentuk kode HTML atau daftar elemen. Melalui visualisasi graf, pengguna dapat melihat hubungan antar elemen, posisi elemen dalam hierarki, serta percabangan struktur halaman secara lebih mudah.

Visualisasi yang dihasilkan perlu memperhatikan keterbacaan graf. Setiap *node* harus memiliki label yang dapat menunjukkan elemen HTML yang direpresentasikan. Arah hubungan antar *node* juga perlu ditampilkan dengan jelas agar pengguna dapat mengetahui elemen induk dan elemen anak. Selain itu, susunan graf perlu dibuat teratur agar struktur halaman tidak terlihat terlalu padat atau membingungkan.

Proses perubahan DOM menjadi format DOT dilakukan berdasarkan hasil penelusuran DFS. Ketika sebuah elemen DOM pertama kali ditemukan, sistem memberikan ID unik kepada elemen tersebut, misalnya `node1`, kemudian menyimpan pasangan antara objek elemen dan ID yang diberikan. Nama *tag* dari elemen digunakan sebagai nilai atribut `label`. Dengan pemisahan antara ID dan label, elemen-elemen yang memiliki nama *tag* sama tetap dinyatakan sebagai simpul yang berbeda. Sebagai contoh, elemen `html` dengan ID `node1` ditulis sebagai deklarasi `node1 [label="html"];` pada format DOT.

Setelah simpul induk dan simpul anak memiliki ID, hubungan langsung keduanya ditulis menggunakan operator panah. Apabila `node1` merupakan induk dari `node2`, hubungan tersebut dinyatakan sebagai `node1 -> node2;`. Setiap deklarasi simpul dan hubungan ditambahkan ke dalam blok graf yang diawali dengan kata kunci `digraph` dan diakhiri dengan kurung kurawal penutup. Karena arah sisi ditulis dari ID elemen induk menuju ID elemen anak, struktur DOT yang terbentuk tetap mempertahankan hierarki DOM hasil *parsing*.

Penyusunan DOT dapat dilakukan bersamaan dengan proses DFS. Sistem terlebih dahulu menambahkan deklarasi untuk elemen yang sedang dikunjungi, kemudian memproses setiap elemen anak, menambahkan deklarasi simpul anak, dan menuliskan *edge* dari induk menuju anak. Apabila label memuat karakter khusus, seperti tanda kutip, garis miring terbalik, atau baris baru, karakter tersebut perlu melalui proses *escaping* agar tidak merusak sintaks DOT. Hasil konversi dapat diperiksa dengan memastikan seluruh elemen memiliki ID unik, setiap relasi langsung *parent-child* menghasilkan satu *edge*, dan elemen akar tidak memiliki sisi masuk. Pada struktur pohon yang terhubung dengan n simpul, jumlah sisi yang terbentuk seharusnya sebanyak $n - 1$.

Dalam penelitian ini, Graphviz digunakan untuk membantu proses visualisasi karena mampu membaca representasi graf dalam format DOT. Oleh karena itu, sistem perlu menghasilkan format DOT yang sesuai dengan struktur DOM yang telah dipetakan. Format tersebut kemudian digunakan untuk membuat visualisasi graf yang dapat diamati oleh pengguna.

3.2 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem dilakukan untuk menentukan fungsi yang harus dimiliki oleh perangkat lunak serta batasan kualitas yang perlu dipenuhi. Berdasarkan judul penelitian, yaitu perangkat lunak pembuat visualisasi graf struktur DOM halaman web, sistem yang dikembangkan berfokus pada proses pengambilan satu halaman web, pembentukan struktur DOM, pemetaan elemen HTML ke dalam bentuk graf, dan penyajian hasil visualisasi kepada pengguna. Graf yang dihasilkan merepresentasikan hubungan *parent-child* pada DOM satu halaman web, bukan hubungan tautan antar halaman.

3.2.1 Analisis *Use Case* dan Skenario

Analisis *use case* digunakan untuk menggambarkan layanan sistem berdasarkan tujuan yang ingin dicapai oleh aktor. Aktor utama pada perangkat lunak ini adalah pengguna yang ingin memperoleh visualisasi graf struktur DOM dari suatu halaman web. Jsoup dan Graphviz tidak diperlakukan sebagai aktor karena keduanya merupakan komponen internal yang digunakan oleh sistem untuk melakukan *parsing* HTML dan menghasilkan visualisasi graf.

Berdasarkan interaksi pengguna dengan sistem, terdapat dua *use case* utama, yaitu membuat visualisasi graf struktur DOM dan melihat detail hasil pemrosesan. Daftar *use case* tersebut ditunjukkan pada Tabel 3.1.

Tabel 3.1: Daftar *use case* sistem

ID	**Nama *Use Case***	**Deskripsi**
UC-01	Membuat visualisasi graf struktur DOM	Pengguna memasukkan URL, kemudian sistem mengambil HTML, membentuk DOM, menelusuri elemen, menghasilkan DOT, dan menampilkan visualisasi graf.
UC-02	Melihat detail hasil pemrosesan	Pengguna melihat daftar *node*, daftar *edge*, dan representasi DOT yang dihasilkan dari proses visualisasi.

Skenario *Use Case* Membuat Visualisasi Graf Struktur DOM

Skenario UC-01 menjelaskan interaksi utama sejak pengguna memberikan URL hingga sistem menampilkan visualisasi graf. Skenario tersebut ditunjukkan pada Tabel 3.2.

Tabel 3.2: Skenario UC-01 membuat visualisasi graf struktur DOM

Bagian	**Keterangan**
Nama *use case*	Membuat visualisasi graf struktur DOM.
Aktor	Pengguna.
Tujuan	Memperoleh visualisasi graf yang merepresentasikan hubungan *parent-child* pada DOM halaman web.
Prakondisi	Aplikasi dapat dijalankan dan perangkat memiliki koneksi untuk mengakses URL tujuan.
Pemicu	Pengguna memasukkan URL dan menjalankan proses visualisasi.
Skenario utama	1. Pengguna memasukkan URL halaman web. 2. Sistem memvalidasi format URL. 3. Sistem mengambil dokumen HTML dari URL menggunakan Jsoup. 4. Sistem melakukan *parsing* HTML menjadi struktur DOM. 5. Sistem menelusuri elemen DOM menggunakan DFS. 6. Sistem memberikan ID unik kepada setiap elemen dan membentuk relasi *parent-child*. 7. Sistem menyusun deklarasi *node* dan *edge* dalam format DOT. 8. Graphviz memproses DOT menjadi visualisasi graf. 9. Sistem menampilkan visualisasi graf kepada pengguna.

Skenario alternatif	a. Jika URL kosong atau formatnya tidak valid, sistem menampilkan pesan kesalahan dan meminta pengguna memperbaiki URL. b. Jika halaman tidak dapat diakses, sistem menampilkan pesan kegagalan koneksi dan menghentikan proses. c. Jika HTML gagal diproses atau DOT gagal divisualisasikan, sistem menampilkan pesan kesalahan sesuai tahap yang gagal.
Pascakondisi	Visualisasi graf berhasil ditampilkan atau sistem memberikan pesan yang menjelaskan penyebab kegagalan.

Skenario *Use Case* Melihat Detail Hasil Pemrosesan

Skenario UC-02 menjelaskan interaksi pengguna ketika melihat informasi pembentuk graf setelah proses visualisasi berhasil dilakukan. Skenario tersebut ditunjukkan pada Tabel 3.3.

Tabel 3.3: Skenario UC-02 melihat detail hasil pemrosesan

Bagian	**Keterangan**
Nama *use case*	Melihat detail hasil pemrosesan.
Aktor	Pengguna.
Tujuan	Memeriksa elemen dan hubungan yang digunakan untuk membentuk visualisasi graf.
Prakondisi	Proses pembentukan DOM dan graf telah berhasil dilakukan.
Pemicu	Pengguna memilih untuk melihat informasi hasil pemrosesan.
Skenario utama	1. Pengguna membuka bagian detail hasil pemrosesan. 2. Sistem menampilkan daftar *node* beserta ID dan label elemen HTML. 3. Sistem menampilkan daftar *edge* yang menunjukkan hubungan induk dan anak. 4. Sistem menampilkan representasi graf dalam format DOT. 5. Pengguna memeriksa kesesuaian detail tersebut dengan visualisasi graf.
Skenario alternatif	Jika data hasil pemrosesan belum tersedia, sistem menampilkan informasi bahwa pengguna harus menjalankan proses visualisasi terlebih dahulu.
Pascakondisi	Informasi *node*, *edge*, dan DOT berhasil ditampilkan tanpa mengubah hasil pemrosesan.

3.2.2 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional merupakan kebutuhan yang berkaitan dengan kualitas, batasan, dan karakteristik sistem dalam menjalankan fungsi-fungsinya. Kebutuhan non-fungsional pada perangkat

lunak ini adalah sebagai berikut:

1. *Usability*: sistem memiliki antarmuka yang mudah digunakan sehingga pengguna dapat memasukkan URL, menjalankan proses visualisasi, dan melihat hasil graf tanpa langkah yang rumit.
2. *Performance*: sistem mampu memproses dokumen HTML dan menghasilkan visualisasi graf dalam waktu yang wajar, terutama untuk halaman web dengan struktur DOM berukuran kecil hingga sedang.
3. *Reliability*: sistem dapat menangani kesalahan yang mungkin terjadi, seperti URL kosong, URL tidak valid, koneksi gagal, atau struktur HTML yang tidak lengkap, tanpa menyebabkan aplikasi berhenti secara tiba-tiba.
4. *Accuracy*: sistem mampu mempertahankan hubungan hierarkis DOM secara tepat ketika mengubah elemen HTML menjadi *node* dan hubungan *parent-child* menjadi *edge*.
5. *Maintainability*: sistem disusun secara terstruktur agar proses pengembangan, perbaikan, dan penambahan fitur dapat dilakukan dengan lebih mudah.
6. *Compatibility*: sistem dapat memproses dokumen HTML dari berbagai situs web selama halaman tersebut dapat diakses dan memiliki struktur HTML yang dapat dibaca oleh pustaka pemrosesan HTML.
7. *Portability*: sistem dapat dijalankan pada lingkungan yang mendukung Java, Jsoup, dan Graphviz sehingga tidak bergantung pada satu perangkat tertentu.
8. *Readability*: hasil visualisasi graf harus memiliki tampilan yang cukup jelas, baik dari sisi penamaan *node*, arah *edge*, maupun susunan graf yang dihasilkan.

3.3 Analisis Sistem Usulan

Sistem usulan pada penelitian ini adalah perangkat lunak yang digunakan untuk membuat visualisasi graf dari struktur DOM satu halaman web. Sistem menerima masukan berupa URL dari pengguna, kemudian mengambil dokumen HTML dari URL tersebut. Dokumen HTML yang berhasil diperoleh diproses menjadi DOM menggunakan pustaka Jsoup. Setelah DOM terbentuk, sistem menelusuri elemen-elemen HTML secara hierarkis dan memetakan setiap elemen menjadi *node* serta hubungan antar elemen menjadi *edge*. Hasil pemetaan tersebut kemudian dikonversi ke dalam format DOT agar dapat divisualisasikan menggunakan Graphviz.

Sistem ini diusulkan untuk membantu pengguna memahami struktur DOM halaman web tanpa harus membaca seluruh kode HTML secara manual. Dengan adanya visualisasi graf, pengguna dapat melihat hubungan induk dan anak antar elemen secara lebih jelas. Selain itu, sistem juga membantu memperlihatkan kedalaman struktur DOM, percabangan elemen, serta susunan elemen pada halaman web dalam bentuk graf berarah.

3.3.1 Gambaran Umum Sistem Usulan

Secara umum, sistem usulan bekerja melalui beberapa tahapan utama, yaitu penerimaan URL, pengambilan dokumen HTML, pembentukan DOM, penelusuran DOM, pembentukan graf, pembuatan format DOT, dan visualisasi graf. Setiap tahapan memiliki peran yang saling berhubungan. Tahap pengambilan HTML menjadi masukan bagi tahap pembentukan DOM, tahap pembentukan

DOM menjadi dasar bagi proses *traversal*, sedangkan hasil *traversal* digunakan untuk membentuk *node* dan *edge* pada graf.

Pada tahap awal, pengguna memasukkan alamat situs web yang akan dianalisis. Sistem kemudian melakukan validasi terhadap URL tersebut. Jika URL valid dan halaman dapat diakses, sistem mengambil dokumen HTML dari halaman tersebut. Jika URL tidak valid atau halaman tidak dapat diakses, sistem menampilkan pesan kesalahan kepada pengguna agar proses dapat diperbaiki.

Setelah dokumen HTML diperoleh, sistem melakukan *parsing* untuk membentuk struktur DOM. Struktur DOM tersebut kemudian ditelusuri menggunakan metode *Depth-First Search* (DFS). Metode DFS digunakan karena sesuai dengan karakteristik DOM yang tersusun secara hierarkis. Dengan DFS, sistem dapat menelusuri elemen dari akar menuju elemen terdalam, kemudian kembali ke cabang lain hingga seluruh elemen selesai diproses.

3.3.2 Model Representasi Graf

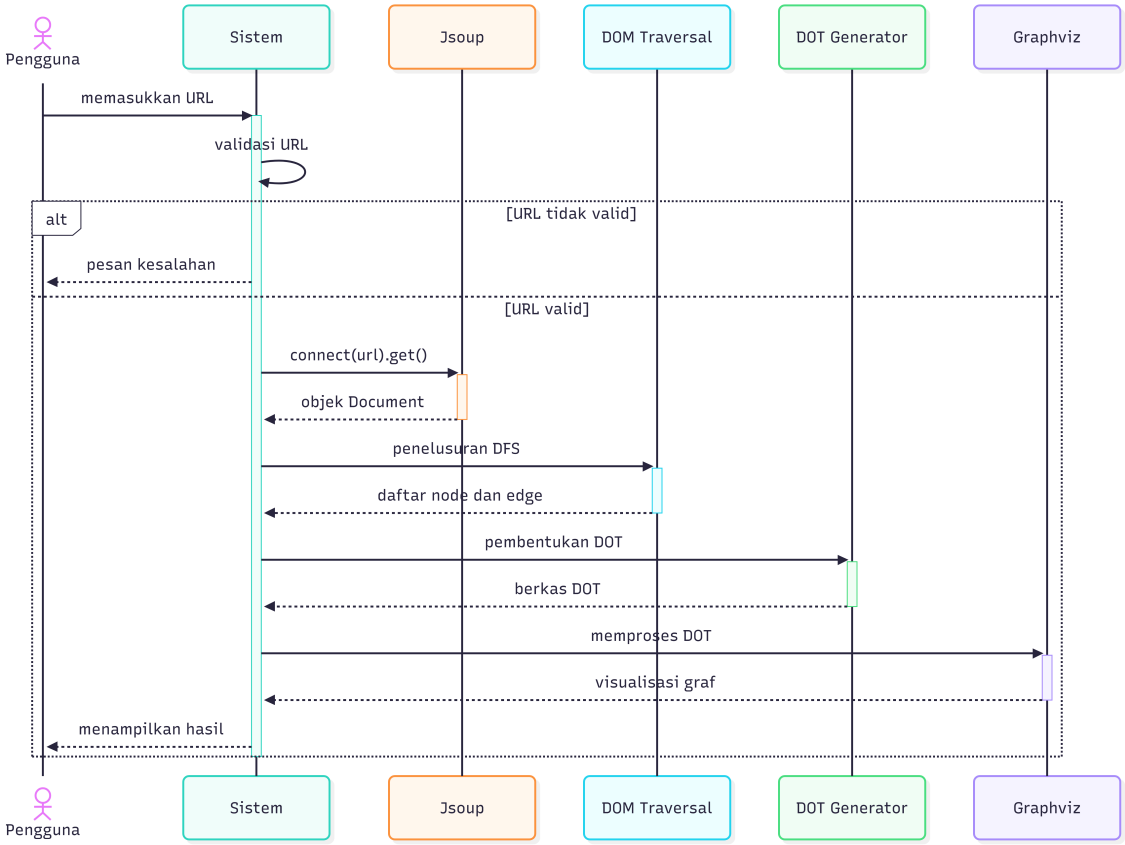
Model representasi graf pada sistem usulan didefinisikan untuk menghindari ambiguitas ketika dokumen HTML memiliki banyak elemen dengan nama *tag* yang sama. Sistem hanya memetakan *element node* pada DOM sebagai simpul graf. *Text node*, *attribute node*, dan *comment node* tidak dijadikan simpul utama karena fokus visualisasi adalah struktur elemen HTML.

Setiap elemen HTML yang ditemukan saat proses *traversal* diberi ID unik, misalnya `node1`, `node2`, dan seterusnya. ID unik tersebut digunakan sebagai identitas simpul pada format DOT, sedangkan nama *tag* HTML digunakan sebagai label yang ditampilkan pada visualisasi. Dengan demikian, beberapa elemen dengan label yang sama, seperti `div` atau `p`, tetap dapat dibedakan oleh sistem karena memiliki ID simpul yang berbeda.

Hubungan antar simpul dibentuk berdasarkan relasi langsung *parent-child* pada DOM. Jika suatu elemen anak berada langsung di dalam elemen induk, maka sistem membuat *edge* berarah dari simpul induk menuju simpul anak. Secara sederhana, jika elemen p adalah induk langsung dari elemen c, maka sistem membentuk sisi berarah (p, c). Model ini membuat graf yang dihasilkan merepresentasikan struktur hierarkis DOM satu halaman web.

3.3.3 Diagram Sekuens Sistem Usulan

Diagram sekuens digunakan untuk menggambarkan urutan interaksi antar komponen ketika pengguna membuat visualisasi graf struktur DOM. Komponen yang terlibat terdiri dari Pengguna, Sistem, Jsoup, *DOM Traversal*, *DOT Generator*, dan Graphviz. Urutan interaksi tersebut ditunjukkan pada Gambar 3.1.



Gambar 3.1: Diagram sekuens proses visualisasi graf struktur DOM

Diagram tersebut dimulai ketika pengguna memasukkan URL dan sistem melakukan validasi. Jika URL tidak valid, sistem menampilkan pesan kesalahan dan proses dihentikan. Jika URL valid, sistem menggunakan Jsoup untuk mengambil HTML dan membentuk objek `Document`. Struktur DOM kemudian ditelusuri menggunakan DFS untuk menghasilkan daftar *node* dan *edge*. Daftar tersebut diubah menjadi format DOT, diproses oleh Graphviz, dan hasil visualisasinya ditampilkan kepada pengguna. Apabila terjadi kegagalan saat pengambilan HTML, *parsing*, pembentukan DOT, atau visualisasi, sistem menampilkan pesan kesalahan sesuai tahap yang gagal.

Interaksi pada diagram hanya memproses halaman berdasarkan URL yang dimasukkan oleh pengguna. Sistem tidak melakukan *web crawling* ke halaman lain sehingga hasil yang dihasilkan tetap berfokus pada struktur DOM satu halaman web.

3.3.4 Komponen Sistem Usulan

Sistem usulan terdiri dari beberapa komponen utama yang mendukung proses visualisasi graf struktur DOM halaman web. Komponen-komponen tersebut adalah sebagai berikut:

1. *Input URL*, yaitu komponen yang digunakan untuk menerima alamat situs web dari pengguna.
2. *HTML Fetcher*, yaitu komponen yang bertugas mengambil dokumen HTML dari URL yang diberikan.
3. *HTML Parser*, yaitu komponen yang memproses dokumen HTML menjadi struktur DOM menggunakan Jsoup.

4. *DOM Traversal*, yaitu komponen yang menelusuri struktur DOM menggunakan metode DFS.
5. *Graph Builder*, yaitu komponen yang membentuk *node* dan *edge* berdasarkan hasil penelusuran DOM.
6. *DOT Generator*, yaitu komponen yang mengubah daftar *node* dan *edge* menjadi format DOT.
7. *Graph Visualizer*, yaitu komponen yang menghasilkan visualisasi graf menggunakan Graphviz.
8. *Output Viewer*, yaitu komponen yang menampilkan hasil visualisasi graf kepada pengguna.

Pembagian komponen tersebut bertujuan agar setiap proses dalam sistem memiliki tanggung jawab yang jelas. Dengan struktur tersebut, sistem lebih mudah dikembangkan, diuji, dan diperbaiki apabila terjadi kesalahan pada salah satu tahapan.

3.3.5 Analisis Masukan dan Keluaran Sistem

Masukan utama pada sistem usulan adalah URL situs web yang diberikan oleh pengguna. URL tersebut digunakan sebagai alamat sumber untuk mengambil dokumen HTML. Selain URL, sistem juga memerlukan dokumen HTML yang berhasil diperoleh dari halaman web sebagai data yang akan diproses pada tahap pembentukan DOM.

Keluaran utama dari sistem adalah visualisasi graf struktur DOM halaman web. Visualisasi tersebut menampilkan elemen-elemen HTML sebagai *node* dan hubungan antar elemen sebagai *edge*. Selain visualisasi graf, sistem juga dapat menghasilkan keluaran antara berupa struktur DOM, daftar *node*, daftar *edge*, dan berkas DOT. Keluaran antara tersebut berguna untuk memastikan bahwa proses pemetaan dari DOM ke graf telah berjalan sesuai dengan struktur DOM halaman web yang dianalisis.

Apabila terjadi kesalahan, keluaran sistem berupa pesan informasi atau pesan kesalahan. Pesan tersebut dapat muncul ketika URL tidak valid, halaman web tidak dapat diakses, dokumen HTML gagal diproses, atau visualisasi graf gagal dibuat. Dengan adanya pesan tersebut, pengguna dapat mengetahui penyebab kegagalan proses dan melakukan perbaikan pada masukan yang diberikan.

DAFTAR REFERENSI

[1] Marini, J. (2002) *The Document Object Model*. McGraw-Hill/Osborne Media, America.

[2] Duckett, J. (2011) *HTML and CSS, Design and Build Websites*. John Wiley and Sons, Inc, Canada.

[3] Hedley, J. dan jsoup contributors (2026) jsoup: Java HTML parser.

[4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., dan Stein, C. (2022) *Introduction to Algorithms*, 4th edition. The MIT Press, America.

[5] Ellson, J., Gansner, E., Hu, Y., North, S., Jacobsson, M., Fernandez, M., dan Hansen, M. (2024) Graphviz. https://graphviz.org/. 28 April 2026.